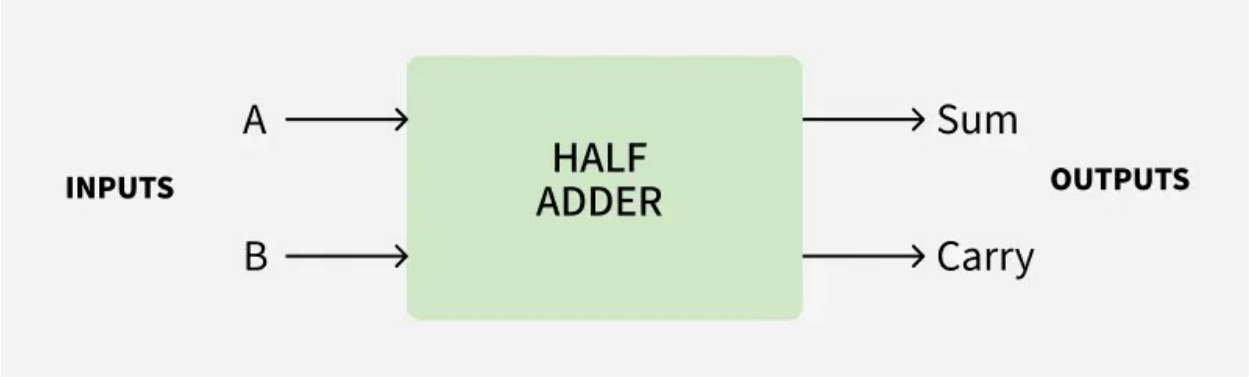
Adders and Subtractors in Digital Logic

Adders and subtractors are digital circuits used to perform arithmetic operations on binary numbers. An adder adds two binary values to produce a sum and often including a carry bit for values that exceed a single digit. A subtractor calculates the difference between two binary numbers, sometimes using methods like two's complement to handle negative results. These circuits form the core of arithmetic logic units (ALUs) in processors and enable efficient calculation and data processing.

Half Adder

- It is a arithmetic combinational logic circuit designed to perform addition of two single bits.
- It contains two inputs and produces two outputs.
- Inputs are called Augend and Added bits and Outputs are called Sum and Carry.



Half Adder

Let us observe the **addition of single bits**,
0+0=0
0+1=1
1+0=1
1+1=10

Since 1+1=10, the result must be two bit output. So, Above can be rewritten as,

0+0=00
0+1=01

1+0=01
1+1=10

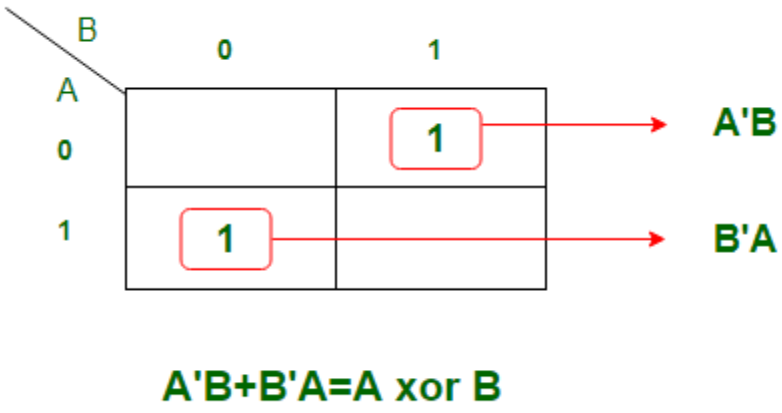
The result of 1+1 is 10, where '1' is carry-output (C_{out}) and '0' is Sum-output (Normal Output).

Truth Table of Half Adder:

Inputs		Outputs	
A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Next Step is to draw the Logic Diagram. To draw Logic Diagram, We need Boolean Expression, which can be obtained using K-map (karnaugh map). Since there are two output variables 'S' and 'C', we need to define K-map for each output variable.

K-map for output variable Sum 'S':



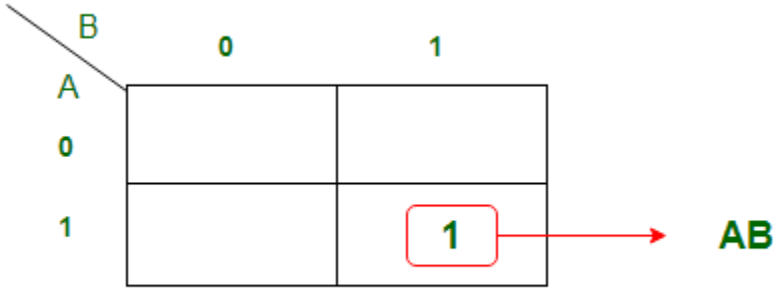
A'B+B'A=A xor B

K-map is of **Sum of products** form. The equation obtained is
S = AB' + A'B

which can be logically written as,

$S = A \oplus B$

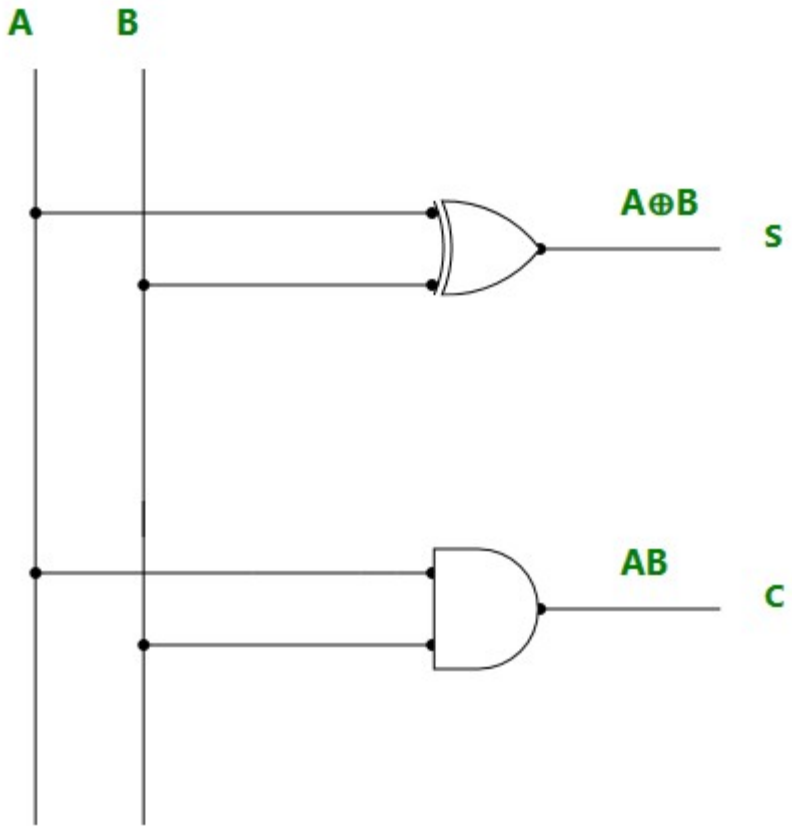
K-map for output variable Carry 'C':



The equation obtained from K-map is,

$C = AB$

Using the Boolean Expression, we can draw logic diagram as follows..

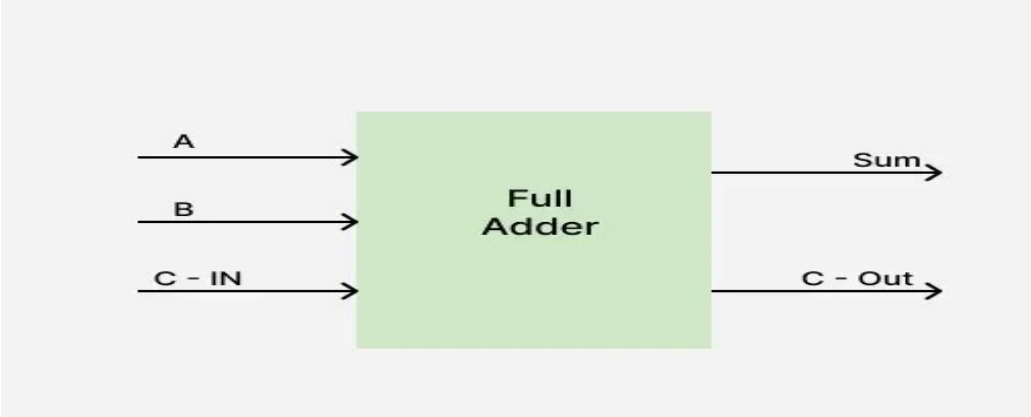


Limitations: Adding of Carry is not possible in Half adder.

Read more about Half Adder.

Full Adder

- To overcome the above limitation faced with Half adders, Full Adders are implemented.
- It is a arithmetic combinational logic circuit that performs addition of three single bits.
- It contains three inputs (A, B, C_{in}) and produces two outputs (Sum and C_{out}).
- Where, C_{in} -> Carry In and C_{out} -> Carry Out

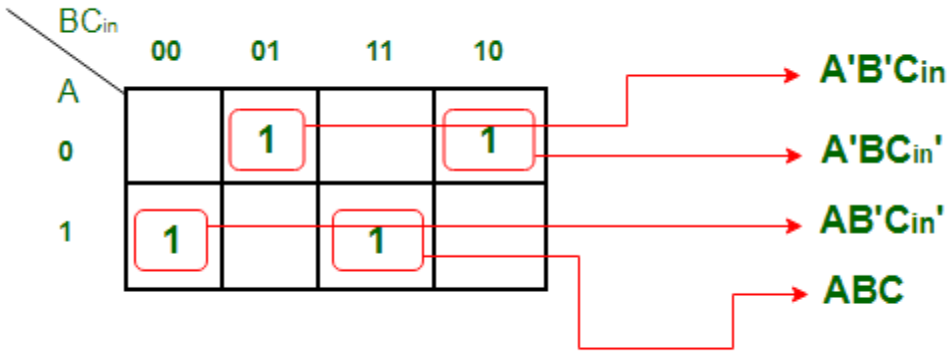


Full Adder

Truth table of Full Adder:

Inputs			Outputs	
A	B	C_{in}	Sum	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

K-map Simplification for output variable Sum 'S' :



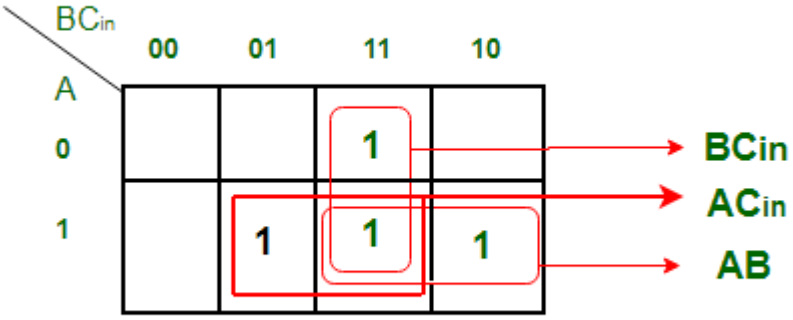
The equation obtained is,

$S = A'B'C_{in} + AB'C_{in}' + ABC + A'BC_{in}'$

The equation can be simplified as,

$S = B'(A'C_{in}+AC_{in}') + B(AC + A'C_{in}')$
$S = B'(A \text{ xor } C_{in}) + B (A \text{ xor } C_{in})'$
$S = A \text{ xor } B \text{ xor } C_{in}$

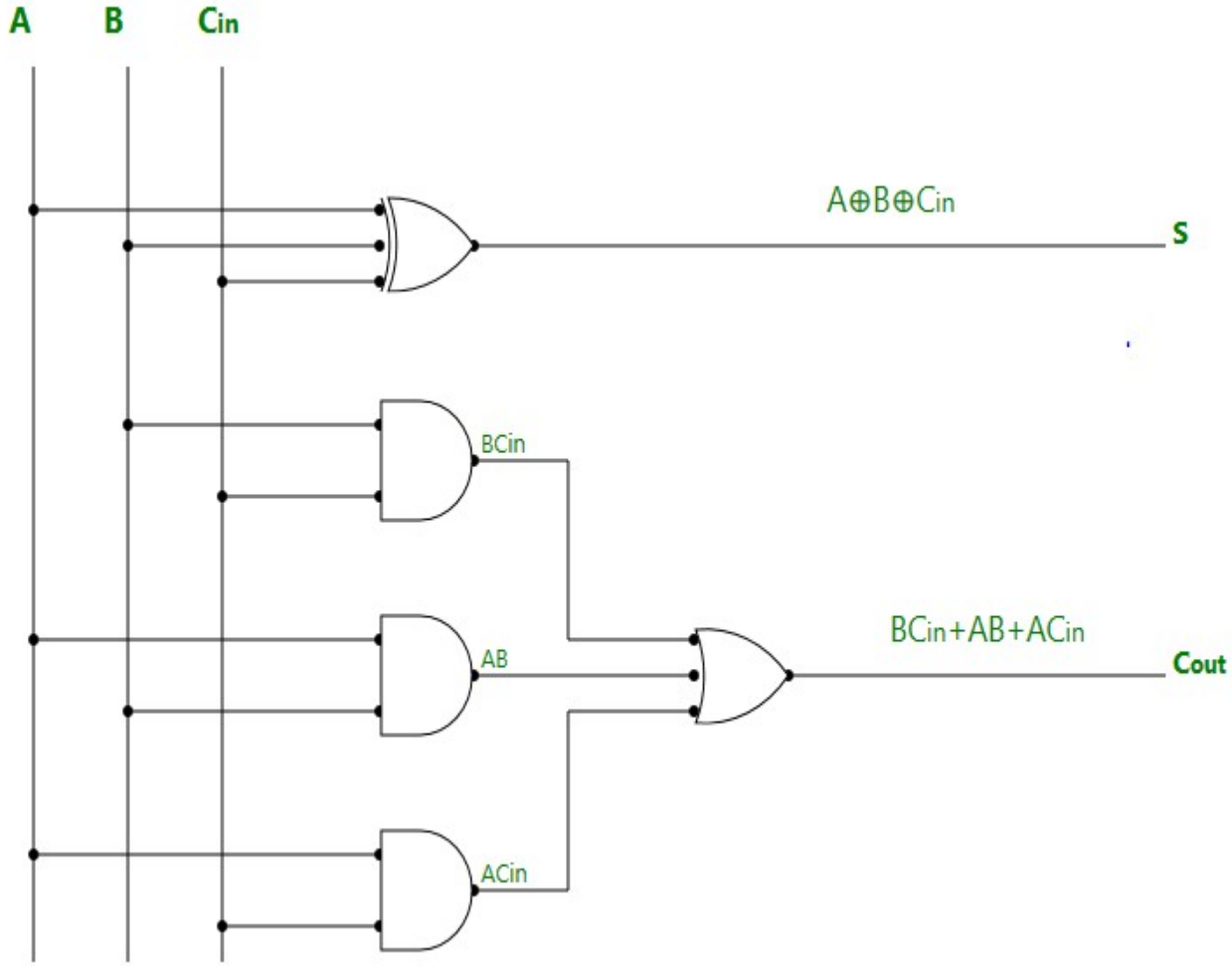
K-map Simplification for output variable 'C_{out}'



The equation obtained is,

$C_{out} = BC_{in} + AB + AC_{in}$

Logic Diagram of Full Adder:



Half Subtractor

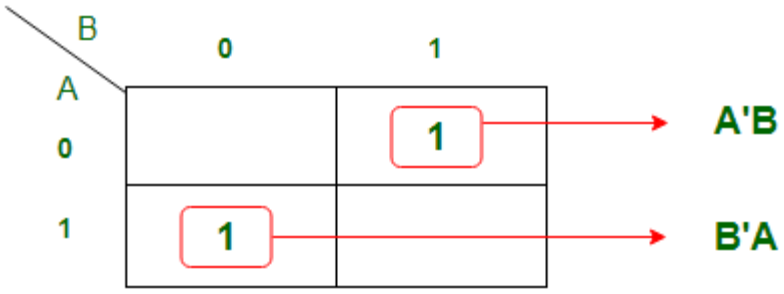
- It is a combinational logic circuit designed to perform the subtraction of two single bits.
- It contains two inputs (A and B) and produces two outputs (Difference and Borrow-output).

Half Subtractor

Truth Table of Half Subtractor:

Inputs		Outputs	
A	B	D	B_0
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

K-map Simplification for output variable 'D':



A'B+B'A=A xor B

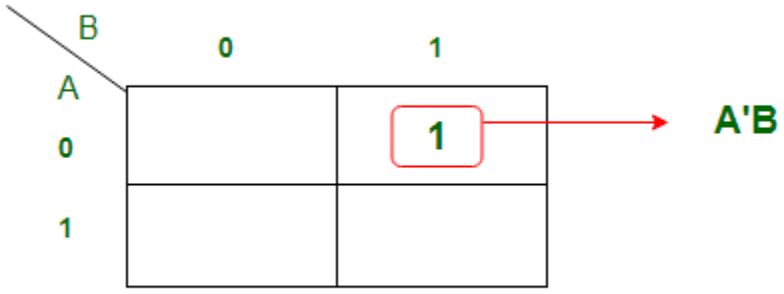
The equation obtained is,

$D = A'B + AB'$

which can be logically written as,

$D = A \text{ xor } B$

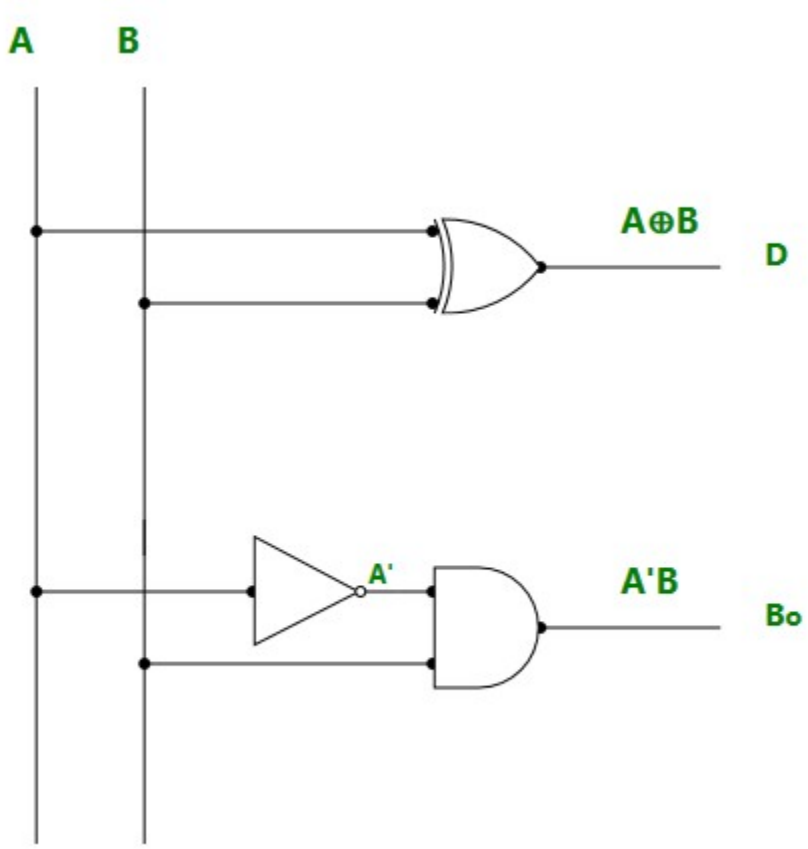
K-map Simplification for output variable 'B_{out}' :



The equation obtained from above K-map is,

$B_{out} = A'B$

Logic Diagram of Half Subtractor:



Read more about Half Subtractor.

Full Subtractor

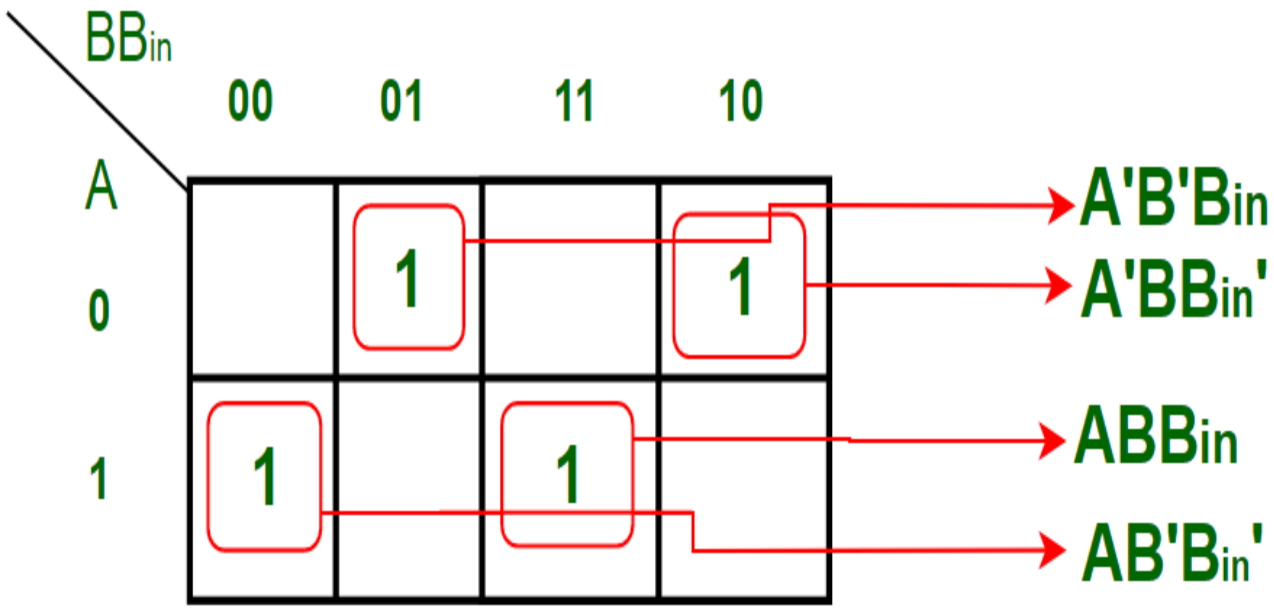
- It is a Combinational logic circuit designed to perform subtraction of three single bits.
- It contains three inputs(A, B, B_{in}) and produces two outputs (D, B_{out}).
- Where, A and B are called **Minuend** and **Subtrahend** bits.
- And, B_{in} -> Borrow-In and B_{out} -> Borrow-Out

Full Subtractor

Truth Table of Full Subtractor:

Inputs			Outputs	
A	B	B_{in}	D	B_{out}
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

K-map Simplification for output variable 'D' :

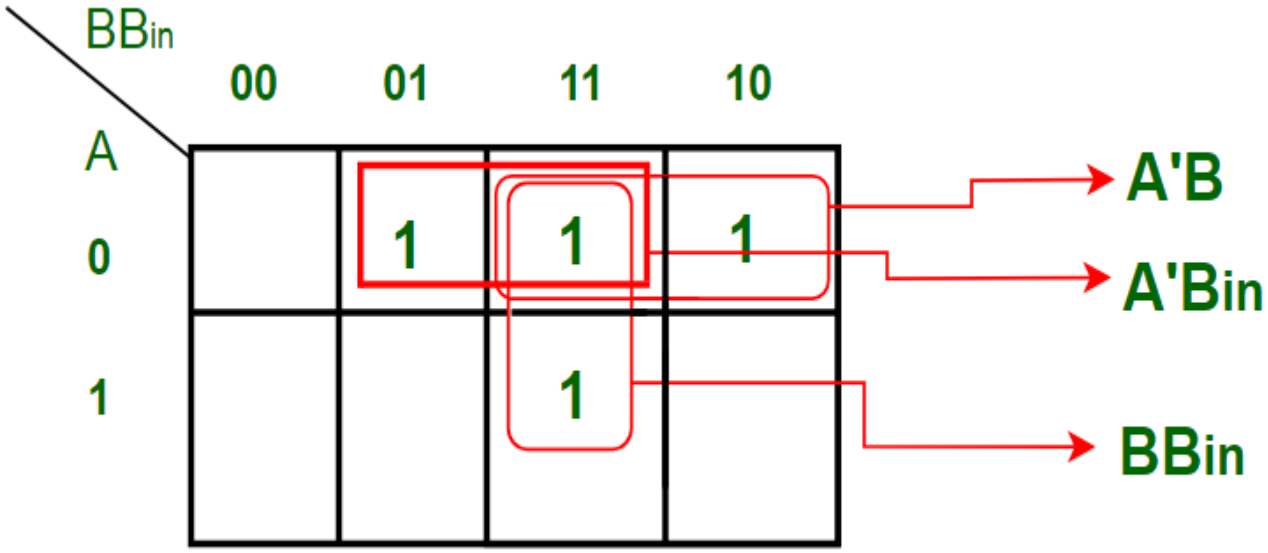


The equation obtained from above K-map is,
$D = A'B'B_{in} + AB'B_{in}' + ABB_{in} + A'BB_{in}'$

which can be simplified as,

$D = B'(A'B_{in} + AB_{in}') + B(AB_{in} + A'B_{in}')$
$D = B'(A \text{ xor } B_{in}) + B(A \text{ xor } B_{in})'$
$D = A \text{ xor } B \text{ xor } B_{in}$

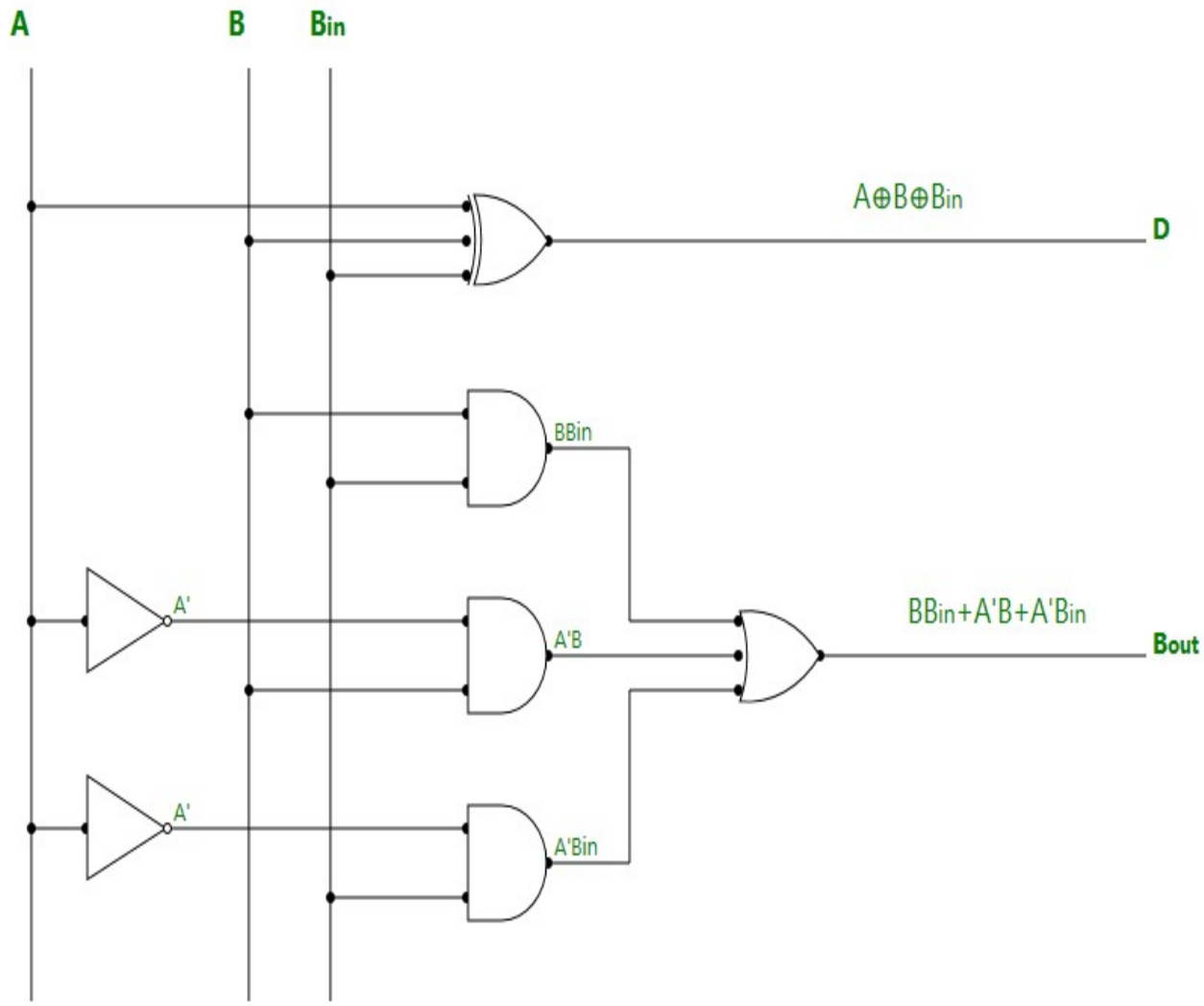
K-map Simplification for output variable 'B_{out}' :



The equation obtained is,

$B_{out} = BB_{in} + A'B + A'B_{in}$

Logic Diagram of Full Subtractor:



Applications:

1. For performing arithmetic calculations in electronic calculators and other digital devices.
2. In Timers and Program Counters.
3. Useful in Digital Signal Processing.

Parallel Adder and Parallel Subtractor

-
-
-

An adder adds two binary numbers one bit at a time using carry from each step. A subtractor subtracts one binary number from another using borrow when needed. A parallel adder adds all bits at once, making addition faster. Similarly, a parallel subtractor subtracts all bits at the same time for quicker results. These are important for fast calculations in computers.

Binary Addition

Binary addition works same as a to regular addition but it only uses two digits: 0 and 1. Remember these rules:

0 + 0 = 0
0 + 1 = 1
1 + 0 = 1
1 + 1 = 10 (which means 0 with a carry of 1)

Example

Add 13 (1 1 0 1) and 11 (1 0 1 1).

1 1 0 1
+ 1 0 1 1
———————
1 1 0 0 0

The result is 11000 in binary, which is 24 in decimal.

Binary Subtraction

Binary subtraction works like normal subtraction but with binary numbers, we mostly use 2’s complement to make subtraction simple.

Steps

- Find the 1’s complement of the the number to be subtracted (change **0 to 1 and 1 to 0**).
- Add 1 in the 1’s complement, and you will get the 2’s complement.
- Add the 2’s complement to the the original number (minuend).
- If there is a carry beyond the bit-length, ignore it.

Example

Subtract 11 (1 0 1 1) from 13 (1 1 0 1).

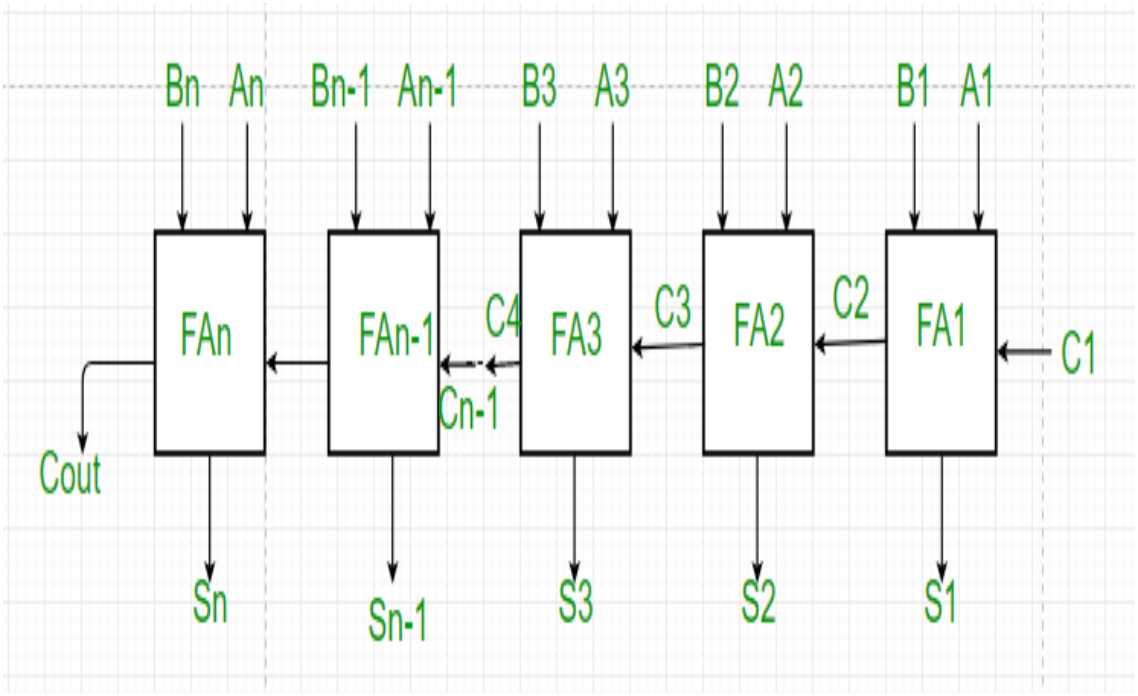
- 1’s complement of 1011 → 0100
- Add 1 → 0101 (2’s complement)
- Add to minuend:

1 1 0 1
+ 0 1 0 1
———
0 0 1 0
Ignore the leftmost carry (1), result is 0010 which is 2 in decimal.

Parallel Adder

A full adder adds two single bits and a carry from the previous addition. It gives two outputs: a sum and a carry. A parallel adder adds two binary numbers that have more than one bit (like 4-bit or 8-bit numbers). It adds all pairs of bits at the same time (in parallel) instead of one after another.

- It is made by connecting many full adders in a row (chain).
- Each full adder handles one pair of bits from the two numbers.
- The carry output from one full adder goes into the carry input of the next full adder on the left (higher bit).
- For an **n-bit number,** you need **n full adders** connected in this way. This design helps add large binary numbers faster than adding bits one by one.



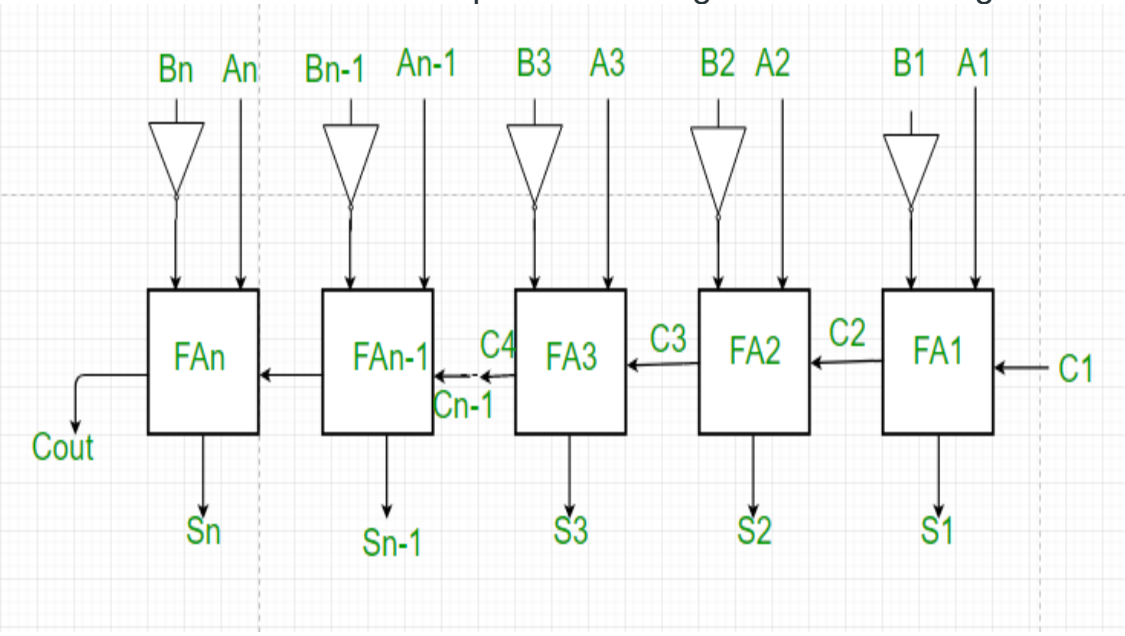
Working of Parallel Adder

1. As you can show in the figure, first of all the full adder FA1 add A1 and B1 along with the carry C1 to generate the sum S1 (the first bit of the output sum) and the carry C2 which is connected to the next adder in chain.
2. Next, the full adder FA2 uses this carry bit C2 to add with the input bits A2 and B2 to generate the sum S2(the second bit of the output sum) and the carry C3 which is again further connected to the next adder in chain and so on.
3. The process continues till the last full adder FAn uses the carry bit Cn to add with its input An and Bn to generate the last bit of the output along last carry bit Cout.

Parallel Subtractor

A Parallel Subtractor is a digital circuit designed to subtract two binary numbers that are more than one bit long by processing pairs of bits simultaneously (in parallel). This allows subtraction to be performed much faster compared to subtracting bit by bit in sequence.

- The parallel subtractor works by subtracting corresponding bits of the two binary numbers at the same time.
- It uses borrow signals similar to how addition circuits use carry signals to handle differences that require borrowing from the next higher bit.



Working of Parallel Subtractor

1. As shown in the figure, the parallel binary subtractor is formed by combination of all full adders with subtrahend complement input.

2. This operation considers that the addition of minuend along with the 2’s complement of the subtrahend is equal to their subtraction.
3. Firstly the 1’s complement of B is obtained by the NOT gate and 1 can be added through the carry to find out the 2’s complement of B. This is further added to A to carry out the arithmetic subtraction.
4. The process continues till the last full adder FAn uses the carry bit Cn to add with its input An and 2’s complement of Bn to generate the last bit of the output along last carry bit Cout.

Advantages of parallel Adder/Subtractor

1. The parallel adder/subtractor performs the addition operation faster as compared to serial adder/subtractor.
2. Time required for addition does not depend on the number of bits.
3. The output is in parallel form i.e. all the bits are added/subtracted at the same time.
4. It is less costly.

Disadvantages of parallel Adder/Subtractor

1. Each adder has to wait for the carry to come from the previous adder in the chain.
2. The propagation delay(delay associated with the travelling of carry bit) is found to increase with the increase in the number of bits to be added

Ripple Counter in Digital Logic

-
-
-

A ripple counter is an asynchronous counter made by cascading flip-flops, where the output of one flip-flop drives the clock of the next. Only the first flip-flop receives the external clock; others are triggered by the previous stage's output, causing a "ripple" effect.

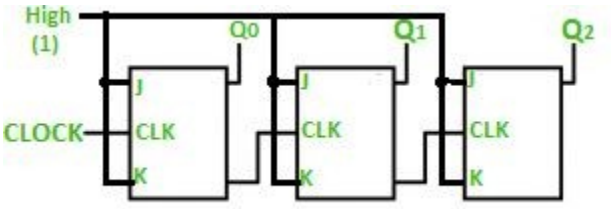
- Asynchronous operation.
- Flip-flops operate in toggle mode.
- Only one flip-flop receives the external clock (LSB).
- Can be configured as up, down, or up/down counter.

An n-bit ripple counter has 2^n states and is also called a MOD-n counter. Counting sequences repeat after reaching the maximum or minimum count:

- Up counter: 000 → 001 → ... → 111 → 000...
- Down counter: 111 → 110 → ... → 000 → 111...

3-bit Ripple counter using a JK flip-flop

In the circuit shown in the below figure, Q0(LSB) will toggle for every clock pulse because JK flip-flop works in toggle mode when both J and K are applied 1, 1, or high input. The following counter will toggle when the previous one changes from 1 to 0.



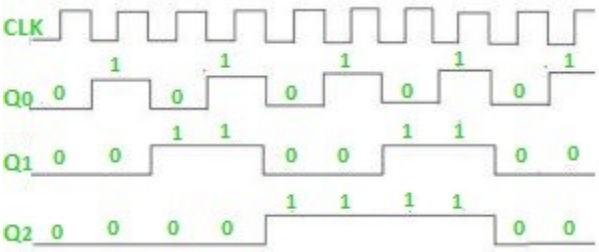
Truth Table

Counter State	Q_2	Q_1	Q_0
0	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0
5	1	0	1
6	1	1	0
7	1	1	1

The 3-bit ripple counter used in the circuit above has eight different states, each one of which represents a count value. Similarly, a counter having n flip-flops can have a maximum of 2 to the power n states. The number of states that a counter owns is known as its mod (modulo) number. Hence a 3-bit counter is a mod-8 counter. A mod-n counter may also be described as a divide-by-n counter. This is because the most significant flip-flop (the furthest flip-flop from the original clock pulse) produces one pulse for every n pulses at the clock input of the least significant flip-flop (the one triggers by the clock pulse). Thus, the above counter is an example of a divide-by-4 counter.

Timing diagram

Let us assume that the clock is negative edge triggered so the above the counter will act as an up counter because the clock is negative edge triggered and output is taken from Q.



Counters are used very frequently to divide clock frequencies and their uses mainly involve digital clocks and in multiplexing. The widely known example of the counter is parallel to serial data conversion logic.

Advantages of Ripple Counter in Digital Logic

- Can be easily designed by T flip-flop or D flip-flop.
- Can be used in low speed circuits & divide by n-counters.
- Used as Truncated counters to design any mode number counters (i.e. Mod 4, Mod 3)

Disadvantages of Ripple Counter in Digital Logic

- Extra flip-flop are needed to do resynchronization.
- To count the sequence of truncated counters, additional feedback logic is needed.
- Propagation delay of asynchronous counters is very large, while counting the large number of bits.
- Counting errors may occur due to propagation delay for high clock frequencies.

Ring Counter in Digital Logic

-
-
-

A ring counter is a typical application of the Shift register. The ring counter is almost the same as the shift counter. The only change is that the output of the last flip-flop is connected to the input of the first flip-flop in the case of the ring counter but in the case of the shift register it is taken as output. Except for this, all the other things are the same.

Ring Counter

It is a type of digital counter used in circuits, typically built with flip-flops. It works by shifting a 1 through a series of flip-flops, one at a time, in a loop.

It’s called a ring counter because the 1 loops back to the start once it reaches the end, forming a cycle.

No. of states in Ring counter = No. of flip-flop used

Key Characteristics of a Ring Counter

- **Single bit '1'**: Only one bit in the counter is set to 1 at any given time.
- **Cyclic Behavior**: The pattern of the bits follows a cyclic behavior and repeats after a fixed number of steps.
- **No Reset**: The ring counter doesn't automatically reset to zero. Instead, it starts at a defined state and then continues cycling.

Working Of Ring Counter

- A Ring Counter is typically built using flip-flops (D, T, or JK flip-flops).
- It consists of n flip-flops, where n is the number of states in the counter. The number of flip-flops determines the number of unique states the counter will cycle through.
- One bit in the counter is set to `1`, and the rest of the bits are set to `0`. In each clock cycle, the `1` bit shifts through the flip-flops, and the pattern repeats.
- The output of the last flip-flop is fed back into the first flip-flop, forming a loop, hence the name Ring Counter.

Example of a 4-bit Ring Counter

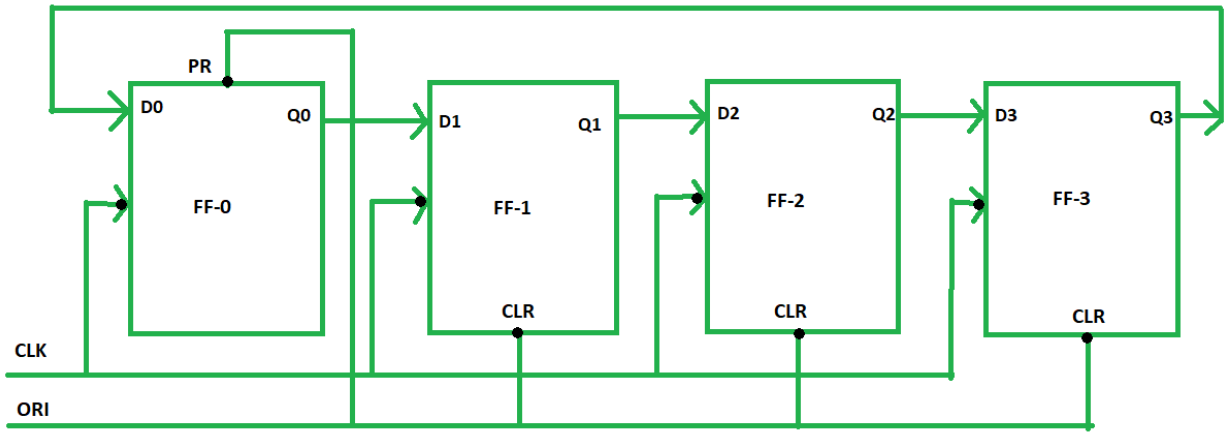
Let’s consider a 4-bit Ring Counter:

1. **Initial state**: `1000` (the first flip-flop is set to 1, and others are set to 0).
2. **Clock cycle 1**: The '1' bit shifts to the next flip-flop, resulting in the state `0100`.
3. **Clock cycle 2**: The '1' bit shifts to the next flip-flop, resulting in the state `0010`.
4. **Clock cycle 3**: The '1' bit shifts to the next flip-flop, resulting in the state `0001`.
5. **Clock cycle 4**: The '1' bit shifts back to the first flip-flop, and the cycle repeats, resulting in the state `1000`.

State Sequence:

`1000 → 0100 → 0010 → 0001 → 1000.`

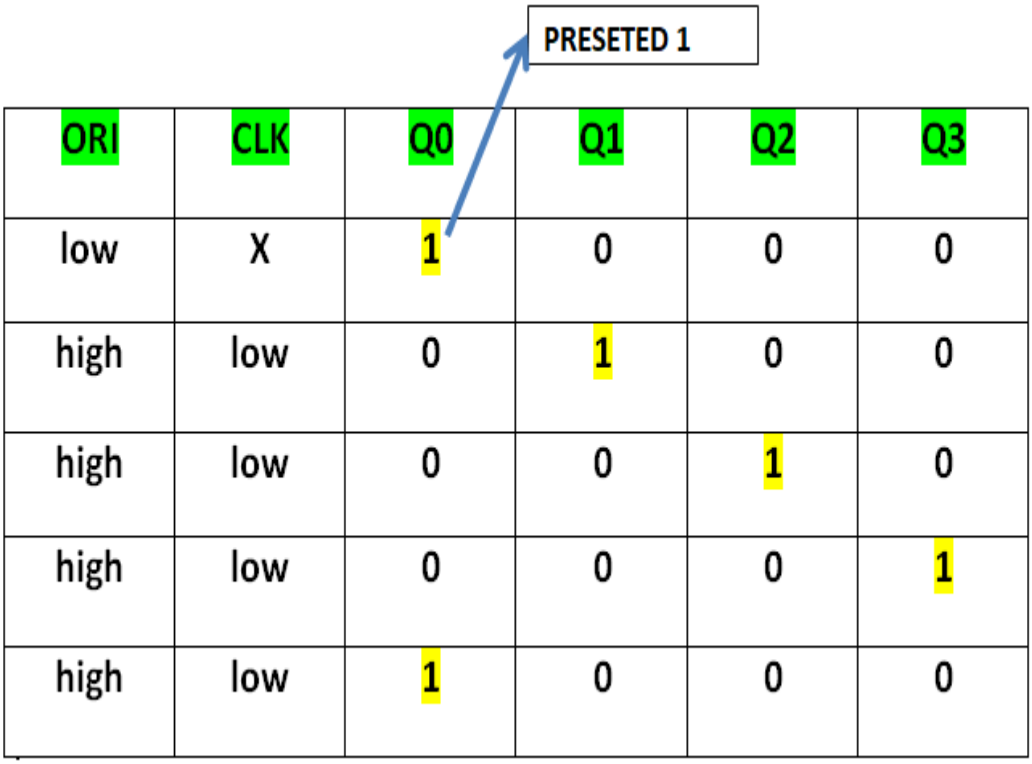
So, for designing a 4-bit Ring counter we need 4 flip-flops as designed below :



Ring Counter

Components and Signals

- **Overriding Input (ORI):**
  The ORI is used to override the normal functionality of the flip-flops. In this case, Preset (PR) and Clear (CLR) are used as ORI.
    - Preset (PR): When the PR signal is `0`, the output of the flip-flop (Q) is set to `1`. This is an active-low signal.
    - Clear (CLR): When the CLR signal is `0`, the output of the flip-flop (Q) is set to `0`. This is also an active-low signal.
- **Preset (PR) = 0, Q = 1**:
  When PR is `0`, the output of the flip-flop is forced to `1`, regardless of other inputs or the clock signal.
- **Clear (CLR) = 0, Q = 0:**
  When CLR is `0`, the output of the flip-flop is forced to `0`, overriding the other inputs.



ORI	CLK	Q0	Q1	Q2	Q3
low	X	1	0	0	0
high	low	0	1	0	0
high	low	0	0	1	0
high	low	0	0	0	1
high	low	1	0	0	0

- The Preset (PR) and Clear (CLR) signals are used to control the state of the flip-flops.
- The ORI input of each flip-flop is connected to the Preset (PR) for FF-0 (the first flip-flop) and to Clear (CLR) for FF-1, FF-2, and FF-3 (the other flip-flops).
    - At FF-0, when PR = 0, the output Q = 1 is generated, and this creates the initial state of the counter.
    - At the other flip-flops (FF-1, FF-2, FF-3), the CLR = 0, forcing the outputs Q = 0 at these flip-flops.
- This setup results in Pre-set 1 at FF-0, and the other flip-flops hold 0. This "1" at FF-0 then propagates through the flip-flops in a cyclic manner, creating the sequence that is characteristic of the Ring Counter.

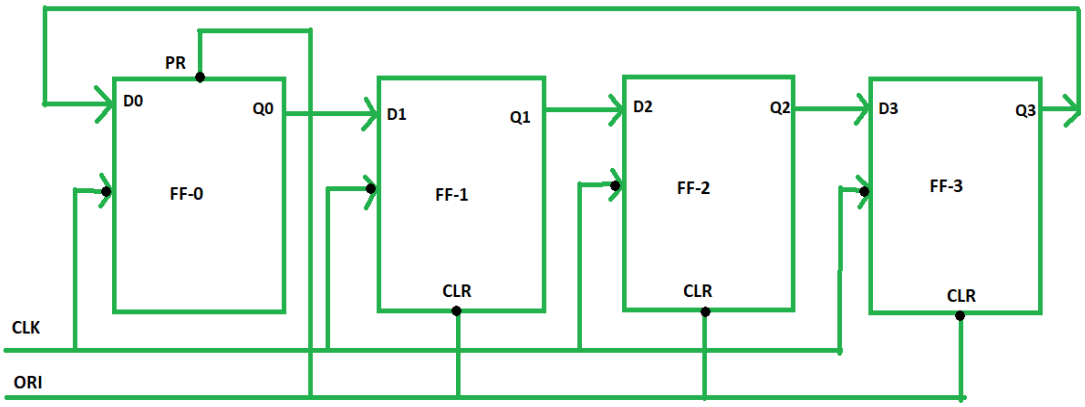
This Preseted 1 is generated by making ORI low and that time Clock (CLK) becomes don't care. After that ORI is made to high and apply low clock pulse signal as the Clock (CLK) is negative edge triggered. After that, at each clock pulse, the preseted 1 is shifted to the next flip-flop and thus forms a Ring. In this way can design a 4-bit Ring Counter using four D flip-flops.

Types of Ring Counter:

There are two types of Ring Counter:

Straight Ring Counter

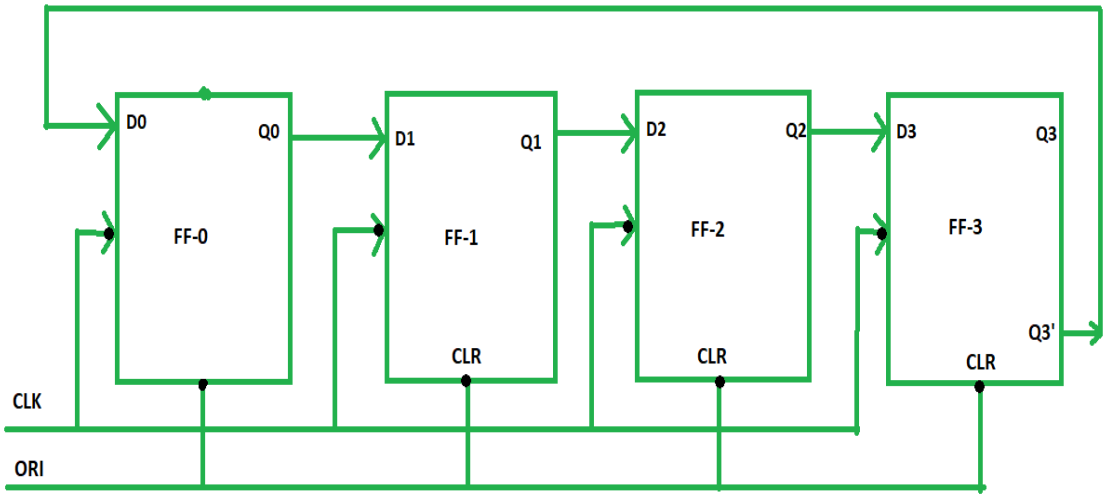
It is also known as One hot Counter. In this counter, the output of the last flip-flop is connected to the input of the first flip-flop. The main point of this Counter is that it circulates a single one (or zero) bit around the ring. Here, we use Preset (PR) in the first flip-flop and Clock (CLK) for the last three flip-flops.



Straight Ring Counter

Twisted Ring Counter

It is also known as a switch-tail ring counter, walking ring counter, or Johnson counter. It connects the complement of the output of the last shift register to the input of the first register and circulates a stream of ones followed by zeros around the ring. Here, we use Clock (CLK) for all the flip-flops. In the Twisted Ring Counter, the number of states = 2 X the number of flip-flops.



Twisted Ring Counter

Advantages of Ring Counter

- **Simplicity**: It's easy to design, especially for small applications.
- **Low Resource Use**: You don't need a lot of components compared to other counters.
- **Predictable Output**: It will always go through the same set of states in the same order.
- **Easy to Decode:** Each state is distinct and easy to decode because it uses one-hot encoding only one output is high at a time.
- **Glitch-Free Output:** Since only one flip-flop is active at a time, there's less chance of output glitches, making it more reliable in certain applications.

Disadvantages of Ring Counter

- **Limited States**: You can only cycle through as many states as the number of flip-flops in the counter. So, for a 4-bit counter, you can only have four unique states.
- **Not for Complex Patterns**: It's not great if you need a counter that counts in random or non-binary patterns since it's limited to the cyclic pattern of shifting one `1` through the flip-flops.
- **Needs Initialization:** A ring counter must be properly initialized (usually with only one flip-flop set to 1) before it starts working. If not, it can get stuck in an invalid state.
- **Hardware Cost Increases:** As you add more flip-flops to increase the number of states, the hardware needed also increases, which can make the design bulky for large state requirements.

Applications of Ring Counter

Ring counters are used when you need a system that goes through a specific, repeating set of states. For example:

- **Sequence Generation**: It’s useful when you want a predictable sequence, like in simple control systems or timing circuits.
- **Control Systems**: Ring counters are great for things like multiplexers or encoder circuits, where the order of operations needs to follow a certain cycle.
- **Simple Design:** Ring counters are easy to design and understand because each state has only one flip-flop set to 1, making debugging and analysis straightforward.
- **Finite State Machines (FSM):** Used in systems that move through a known set of states in a loop, such as vending machines or elevator controllers.
- **Shift Register Based Applications:** Ring counters are a special type of shift register, so they are used where shift registers are needed for data movement.